Guía del laboratorio — OWASP Amass

Laboratorio de reconocimiento y mapeo de superficie de ataque Dominio ficticio: `lab.local` · Red: `192.168.56.0/24`

Expositor: Michael Stiven Tabares Tobón **Curso:** Seguridad de la Información — 2026-II · Ingeniería de Sistemas, UdeA

Aviso

Todo este laboratorio corre dentro de tu portátil:

- El dominio `lab.local` **no existe en Internet** (TLD reservado por la IANA).
- La red `192.168.56.0/24` es **privada** (RFC 1918).
- El servidor DNS es un contenedor de Docker en tu máquina.

No se puede hacer daño a nadie con este laboratorio.

Al final de la guía hay dos comandos para practicar reconocimiento **pasivo** contra un dominio real, que no tocan el objetivo.

Las dos máquinas

Máquina	Qué hace	Cómo entras
Tu portátil	Escribes los comandos	Ya estás aquí
Amass	La herramienta y su base de datos	`docker compose exec amass bash`

Regla: **todos los comandos de `amass` se ejecutan dentro de la máquina `amass`**. Para entrar:

```
cd lab-infra/docker
docker compose -f compose.yaml exec amass bash
```

Para salir: `exit`.

Paso 1 · Montar el laboratorio

Desde la raíz del proyecto:

```
./start.sh
```

La primera vez tarda unos 2 minutos (descarga y construye las imágenes).

Salida esperada:

```
==> Verificando requisitos
 [✓] Docker: Docker version 29.7.2
 [✓] Docker Compose: 5.5.1
 [✓] Daemon de Docker accesible

==> Construyendo las imágenes (la primera vez tarda ~2 min)
 [✓] Imágenes construidas

==> Levantando el laboratorio
 [✓] Contenedores iniciados

==> Esperando a que el motor de Amass esté listo
 [✓] Motor listo tras 1s

==> Verificando el DNS del laboratorio
 [✓] DNS del laboratorio responde (192.168.56.101:53)
 [✓] Resolución de subdominios funciona (dev.lab.local -> 192.168.56.103)
 [✓] NXDOMAIN correcto para los candidatos inexistentes
```

Para empezar de cero:

```
./start.sh --reset
```

Paso 2 · Entrar en la máquina Amass

```
cd lab-infra/docker
docker compose -f compose.yaml exec amass bash
```

El prompt cambia. A partir de aquí estás dentro del contenedor y todos los comandos se ejecutan en la máquina Amass.

Para comprobarlo:

```
hostname        # deve mostrar algo como amass-lab-amass
which amass    # /bin/amass
```

Para salir: `exit`.

Paso 3 · Comandos de la práctica

Todos estos se ejecutan **dentro de la máquina Amass** (después del Paso 2).

Ver la versión

```
amass -version
```

```
v5.1.3
```

Ver la ayuda

```
amass -h
amass enum -h
amass subs -h
```

Comprobar que el DNS ficticio responde

Un nombre que sí existe:

```
dig +short @192.168.56.101 dev.lab.local A
```

```
192.168.56.103
```

Un nombre que no existe (el DNS responde NXDOMAIN, no hay salida):

```
dig +short @192.168.56.101 test.lab.local A
```

Enumeración activa con fuerza bruta

```
amass enum \
  -engine http://127.0.0.1:4000 \
  -active \
  -brute -w /wl/subdomains-lab.txt \
  -d lab.local \
  -p 80,443 \
  -timeout 3
```

Bandera	Significado
`-engine http://127.0.0.1:4000`	Habla con el motor que corre en esta máquina
`-active`	Pregunta al DNS y comprueba los puertos
`-brute -w /wl/subdomains-lab.txt`	Prueba los nombres de esa lista
`-d lab.local`	El objetivo
`-p 80,443`	Solo mira estos dos puertos
`-timeout 3`	Si deja de avanzar 3 minutos, se detiene

Tarda entre 2 y 4 minutos.

Ver los resultados

```
amass subs -names -d lab.local -ip
```

```
admin.lab.local 192.168.56.105
api.lab.local 192.168.56.109
dev.lab.local 192.168.56.103
gitlab.lab.local 192.168.56.107
grafana.lab.local 192.168.56.108
jenkins.lab.local 192.168.56.107
lab.local 192.168.56.103
mail.lab.local 192.168.56.103
ns1.lab.local 192.168.56.101
old.lab.local 192.168.56.110
qa.lab.local 192.168.56.104
staging.lab.local 192.168.56.103
vpn.lab.local 192.168.56.106
www.lab.local 192.168.56.103
```

Fuerza bruta completa (recomendado para la demo)

Este script hace la enumeración base y luego prueba cada candidato del wordlist por separado, porque en Amass v5.1.3 la opción `-nf` entra en panic si se le pasan varios nombres a la vez.

Sal de la máquina Amass (`exit`) y ejecuta desde la raíz del proyecto:

```
./start.sh --brute
```

Salida:

```
==> Fuerza bruta DNS · 13 candidatos · dominio lab.local
==> Paso 1/2 · Enumeración base del dominio
 [✓] Motor listo
==> Paso 2/2 · Validando cada candidato contra el DNS
    www.lab.local [✓] EXISTE
    mail.lab.local [✓] EXISTE
    ...
    test.lab.local [✗] descartado (NXDOMAIN)
    stage.lab.local [✗] descartado (NXDOMAIN)

==> Resultado de la fuerza bruta
    Candidatos probados : 13
    Confirmados         : 11
    Descartados         : 2
```

Esa diferencia es el punto: **la fuerza bruta no solo confirma, también descarta**. El DNS es el que decide.

Ver las consultas DNS registradas

Sal de la máquina Amass (`exit`) y desde `lab-infra/docker` :

```
docker compose -f compose.yaml logs dns | grep "query" \
  | sed 's/.*\[query\] //' | awk '{print $1}' | sort -u
```

Inspeccionar la base de datos de activos

Dentro de la máquina Amass:

```
sqlite3 /.config/amass/asset.db ".tables"
```

```
edge  entity  entity_tag  fqdn  ipaddress  service ...
```

Los nombres:

```
sqlite3 /.config/amass/asset.db "SELECT fqdn FROM fqdn ORDER BY fqdn;"
```

Las IPs:

```
sqlite3 /.config/amass/asset.db "SELECT ip_address FROM ipaddress;"
```

Las relaciones del grafo (qué activo se relaciona con cuál):

```
sqlite3 -header -column /.config/amass/asset.db "
SELECT f.fqdn AS origen, e.label AS relacion, ip.ip_address AS destino
FROM edge e
JOIN entity se ON se.entity_id = e.from_entity_id
JOIN fqdn f ON f.id = se.row_id
JOIN entity de ON de.entity_id = e.to_entity_id
LEFT JOIN ipaddress ip ON ip.id = de.row_id
ORDER BY f.fqdn LIMIT 12;"
```

```
     origen         relacion      destino
----------------  ----------  -----------------
admin.lab.local    dns_record  192.168.56.105
api.lab.local      dns_record  192.168.56.109
dev.lab.local      dns_record  192.168.56.103
```

Ver la zona DNS completa

Sal de la máquina Amass (`exit`) y desde `lab-infra/docker` :

```
docker compose -f compose.yaml exec dns cat /app/zone.txt
```

Paso 4 · Resumen

Acción	Dónde
Montar, reset, detener	Portátil: `./start.sh`, `--reset`, `--stop`
Entrar a la máquina Amass	Portátil: `cd lab-infra/docker && docker compose -f compose.yaml exec amass bash`
Todo lo de Amass, dig y sqlite3	Dentro de la máquina Amass
Logs del DNS y la zona	Portátil: `docker compose -f compose.yaml logs dns`, `exec dns cat /app/zone.txt`

Atajos del script

Si no quieres escribir comandos de Docker:

Comando	Qué hace
`./start.sh`	Monta el laboratorio y verifica el DNS
`./start.sh --reset`	Empieza de cero
`./start.sh --enum`	Ejecuta la enumeración activa
`./start.sh --brute`	Fuerza bruta completa (confirma y descarta)
`./start.sh --results`	Muestra los resultados guardados
`./start.sh --dns`	Muestra las consultas DNS registradas
`./start.sh --check`	Verifica el estado
`./start.sh --logs`	Logs en vivo
`./start.sh --stop`	Detiene el laboratorio
`./start.sh amass <args>`	Ejecuta un comando de Amass a mano

```
./start.sh amass -version
./start.sh amass subs -names -d lab.local -ip
```

La zona del laboratorio

`lab.local` tiene 32 nombres en `192.168.56.0/24`. Los que importan:

Nombres	IP	Severidad
`vpn`	192.168.56.106	Crítico
`admin` , `portal`	192.168.56.105	Alto
`jenkins` , `gitlab`	192.168.56.107	Alto
`dev` , `staging`	192.168.56.103	Alto
`qa` , `uat` , `preprod`	192.168.56.104	Alto
`grafana` , `kibana`	192.168.56.108	Medio
`api` , `beta` , `sandbox`	192.168.56.109	Medio
`old` , `backup`	192.168.56.110	Medio
`www` , `mail`	192.168.56.103	Bajo
`ns1`	192.168.56.101	Infraestructura

Además hay 8 alias CNAME (`intranet` , `secure` , `ci` , `scm` , `metrics` , `logs` , `preview` , `legacy`) y un registro SRV (`_sip._tcp`).

Amass no prioriza: entrega el inventario con su procedencia. La severidad depende del contexto, y eso lo decide el equipo de seguridad, no el script.

Dos comandos pasivos contra un dominio real

Esta es la parte que sí se puede hacer en Internet sin autorización, porque no se toca el objetivo: solo se consultan fuentes de inteligencia abierta.

Ejecuta estos en tu máquina, fuera del laboratorio.

1. Reconocimiento pasivo con Amass

```
amass enum -d example.com -timeout 2
```

Consulta Certificate Transparency, Wayback, RapidDNS y otras fuentes públicas. No genera tráfico hacia el objetivo. Es la técnica de la fase de reconocimiento inicial de una auditoría, y la que la mayoría de programas de bug bounty permiten.

Tarda varios minutos: cada plugin externo espera su propio timeout.

2. Certificate Transparency sin Amass

```
curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '.[].name_value' | sort -u
```

Certificate Transparency es un registro público en el que las autoridades de certificado publican cada certificado que emiten. Por eso es la mejor fuente pasiva de subdominios: revela nombres que la organización no publica en ningún sitio.

Qué no hacer sin autorización

Técnica	Por qué
Fuerza bruta DNS contra un dominio ajeno	Miles de consultas a servidores que no son tuyos
Escaneo de puertos	Sondea infraestructura de terceros
Transferencia de zona (AXFR)	Aunque falle, es tráfico no autorizado

Que un dominio "no tenga nada interesante" no lo convierte en un objetivo permitido. En Colombia, escanear sin autorización es un delito informático (Ley 1273 de 2009 y Decreto 175 de 2015).

Problemas frecuentes

Síntoma	Causa	Solución
`Failed to perform a bulk transfer of assets: EOF`	Bug de Amass v5.1.3 en el endpoint `bulk` de la API. Mata el motor.	No uses `-nf` con varios nombres. Usa `./start.sh --brute`, que los pasa de a uno.
`Failed to create a session with the Amass engine`	El motor no está listo	`./start.sh` espera al motor. Si persiste, `--reset`.
`Failed to obtain the BGPTools IP address`	`bgp.tools` no resuelve en el DNS del laboratorio	Verifica que está en `lab-infra/docker/zone.txt` y haz `--reset`
`flag provided but not defined: -src / -ip / -o`	Banderas de v3/v4 que ya no existen en v5	Usa `amass subs` para leer resultados
`subcommand provided but not defined: db`	El subcomando `db` fue eliminado en v5	Usa `amass subs` o `sqlite3`
La enumeración no descubre nada	Faltan las `transformations` en el config	Restaura el bloque `transformations:` de `lab-infra/configs/config.yaml`
`service "amass" is not running`	Los contenedores están detenidos	`./start.sh` los levanta
`Address already in use`	Ya hay contenedores con esos nombres	`./start.sh --stop` y vuelve a levantar
`Failed to populate the ASN cache: zero entities found`	Normal: requiere WHOIS/RDAP	Ignorar

Síntoma	Causa	Solución
`docker: permission denied`	Tu usuario no está en el grupo `docker`	`sudo usermod -aG docker $USER` y reinicia sesión

Instalar Amass en tu equipo (opcional)

No hace falta para el laboratorio. Si quieres practicar la herramienta fuera del laboratorio:

Arch Linux, con el binario oficial:

```
cd /tmp
curl -LO https://github.com/owasp-amass/amass/releases/download/v5.1.3/amass_linux_amd64.tar.gz
curl -LO https://github.com/owasp-amass/amass/releases/download/v5.1.3/amass_checksums.txt
sha256sum -c amass_checksums.txt --ignore-missing
tar xzf amass_linux_amd64.tar.gz
sudo install -m 755 amass_linux_amd64/amass /usr/local/bin/amass
```

Kali Linux, desde los repositorios:

```
sudo apt update
sudo apt install -y amass
```

Comprobar:

```
amass -version
```

> Con Amass instalado de forma nativa sí se consulta Internet, porque las fuentes OSINT son externas. En el laboratorio no.

Referencias

- Sitio del proyecto: https://owasp.org/projects/amass
- Documentación oficial v5: https://owasp-amass.github.io/docs/
- Código fuente: https://github.com/owasp-amass/amass

Guía validada con OWASP Amass v5.1.3 sobre Docker.